DOCUMENTACIÓN TÉCNICA

Salud Vacunas

Cómo funcionan los datos del proyecto, la carpeta `src` y la modificación del código.

PROYECTO
Pro2 · Sistema de vacunación

STACK
HTML, CSS, JavaScript nativo, Tailwind CSS y daisyUI

ALCANCE
Frontend estático, datos de demostración y sesión del navegador

ACTUALIZADO
5 de noviembre de 2026

CONTENIDO

Índice

01 Visión general del proyecto

02 Estructura de carpetas y archivos

03 Cómo se carga y se inicializa una página

04 El sistema de datos con atributos `data-*`

05 La carpeta `src` : JavaScript y CSS

06 Cómo modificar apartados del código

07 Sesión, persistencia y modelo de datos

08 Relación con `esquema.sql`

09 Errores frecuentes y lista de verificación

10 Referencia rápida

SECCIÓN 01

Visión general del proyecto

Salud Vacunas es una aplicación web estática con tres perfiles. No usa un framework de interfaz ni un servidor: cada vista es una página HTML independiente que carga JavaScript clásico y un CSS compilado.

Qué hace la aplicación

- **Perfil 1 · Centro de salud:** registra usuarios y vacunaciones, con peso, altura, vacuna, lote y vencimiento.
- **Perfil 2 · Sub-región:** consulta el historial de certificados médicos y permite registrar nuevos certificados.
- **Perfil 3 · Master:** consulta la red completa: historial de vacunaciones, certificados, centros, perfiles y avisos.
- **Páginas públicas:** inicio de sesión de demostración, recuperación de contraseña y verificación de certificados.

Cómo se abre

El punto de entrada es `index.html`. Se puede abrir directamente con doble clic; no hace falta instalar un servidor. La aplicación está pensada para funcionar con `file://`, por eso no usa módulos ES ni importaciones entre archivos JavaScript.

> **Importante:** los datos que se crean mientras se navega no se guardan en una base de datos. Se guardan en la sesión del navegador y se pierden al cerrar la pestaña o el navegador. La base de datos solo está definida en `esquema.sql`; todavía no está conectada a la interfaz.

Qué significa que sea estática

- No hay `fetch` a una API ni endpoints de backend.
- Los datos de demostración están escritos en bloques ocultos dentro del HTML.
- Los formularios leen y escriben en el DOM y en la sesión del navegador.
- El CSS se compila con Tailwind CSS y daisyUI mediante npm.

SECCIÓN 02

Estructura de carpetas y archivos

Cada carpeta agrupa una responsabilidad concreta. Las páginas de los perfiles están separadas por perfil para que cada una sea autónoma y fácil de abrir directamente.

```
Pro2/
├─ index.html
├─ recuperar-contrasena.html
├─ verificar.html
├─ assets/
│  ├─ css/estilos.css
│  ├─ fuentes/
│  ├─ favicon.svg
│  └─ afiche-certificado-medico.svg
├─ src/
│  ├─ css/estilos-fuente.css
│  └─ js/
│     ├─ nucleo.js
│     ├─ datos.js
│     ├─ comun.js
│     ├─ publico.js
│     ├─ historial.js
│     ├─ consulta.js
│     ├─ certificados.js
│     ├─ centro.js
│     ├─ avisos.js
│     └─ app.js
├─ master/
│  ├─ historial.html
│  ├─ certificados.html
│  ├─ centros.html
│  ├─ consulta.html
│  └─ avisos.html
├─ subregion/
│  ├─ certificados-historial.html
│  └─ certificados.html
├─ centro/
│  ├─ usuarios/registrar.html
│  └─ vacunas/registrar.html
├─ esquema.sql
├─ INSTRUCCIONES.md
├─ README.md
├─ package.json
└─ node_modules/
```

Carpeta o archivo	Para qué sirve	Qué se modifica allí
`index.html`	Inicio de sesión de demostración y accesos a los tres perfiles.	Textos, accesos, estructura de la portada.
`recuperar-contrasena.html`	Formulario visual de recuperación de contraseña.	Campos y mensajes del formulario.
`verificar.html`	Verificación pública de certificados por código.	Formulario de búsqueda y presentación del resultado.
`master/`	Vistas del Perfil 3: historial, certificados, centros, consulta y avisos.	HTML de cada vista y sus datos ocultos.
`subregion/`	Vistas del Perfil 2: historial de certificados y registro de certificados.	HTML de las dos vistas y sus datos ocultos.
`centro/`	Vistas del Perfil 1: registrar usuario y registrar vacunación.	Formularios y datos del centro.
`src/js/`	Código JavaScript fuente de toda la aplicación.	Comportamiento, validaciones, lectura de datos y renderizado.
`src/css/estilos-fuente.css`	Fuente del CSS: temas, componentes, iconos y estilos de impresión.	Colores, tipografía, iconos, reglas globales.
`assets/css/estilos.css`	CSS compilado que cargan las páginas.	No se edita a mano: se genera con `npm run css`.
`assets/fuentes/`	Archivos de la tipografía Inter Variable.	Solo si se cambia la tipografía.
`assets/`	Favicon, afiche descargable y recursos estáticos.	Imágenes, SVG, documentos descargables.
`esquema.sql`	Esquema PostgreSQL del modelo de datos.	Tablas, columnas, claves foráneas e índices.
`INSTRUCCIONES.md`	Especificación funcional de las vistas.	Requisitos y comportamientos esperados.
`README.md`	Guía breve de uso y estructura.	Instrucciones generales del proyecto.
`package.json`	Dependencias y comandos de compilación del CSS.	Versiones y scripts npm.
`node_modules/`	Dependencias instaladas por npm.	No se edita manualmente.

> **Regla práctica:** si el cambio es visual o de texto, empieza por el HTML de la vista. Si el cambio es de comportamiento, validación o cálculo, ve a `src/js/`. Si el cambio es de color, tipografía o icono, ve a `src/css/estilos-fuente.css` y recompila.

SECCIÓN 03

Cómo se carga y se inicializa una página

Todas las vistas cargan los scripts en un orden concreto. Ese orden es importante: cada archivo depende de funciones que han sido publicadas antes en el espacio global `window.Pro2`.

Orden típico de scripts

```
<script src="../src/js/nucleo.js"></script>
<script src="../src/js/datos.js"></script>
<script src="../src/js/comun.js"></script>
<script src="../src/js/publico.js"></script>
<script src="../src/js/certificados.js"></script>
<script src="../src/js/app.js"></script>
```

Archivo	Qué aporta	Se necesita antes de...
`nucleo.js`	Utilidades, tema, sesión, alertas y constantes.	Cualquier otro módulo.
`datos.js`	Lectores de datos: usuarios, registros, vacunas, centros y certificados.	Los módulos que consultan datos.
`comun.js`	Fichas de usuario, esquema de vacunación y HTML de certificados.	Consulta y certificados.
`publico.js`	Navegación, salida, tema, login, verificación y recuperación.	`app.js`.
`módulo de vista`	Lógica propia de la vista: historial, centro, certificados, avisos, etc.	`app.js`.
`app.js`	Ejecuta la navegación y el inicializador de la página.	Al final de todos.

El atributo `data-pagina`

Cada `<body>` declara qué vista es. Por ejemplo:

```
<body data-pagina="centro-registrar-vacunacion">
```

`app.js` usa ese valor para buscar el inicializador correcto:

```
const paginas = {
  login: [Pro2.iniciarLogin],
  recuperar: [Pro2.iniciarRecuperar],
  verificar: [Pro2.iniciarVerificar],
  "centro-registrar-usuario": [Pro2.iniciarRegistroUsuario],
  "centro-registrar-vacunacion": [Pro2.iniciarVacunacion],
  "master-historial": [Pro2.iniciarHistorial],
  "master-certificados": [Pro2.iniciarHistorialCertificados],
  "master-centros": [Pro2.iniciarCentros],
  "master-avisos": [Pro2.iniciarAvisos],
  "subregion-certificados": [Pro2.iniciarCertificados],
  "subregion-certificados-historial": [Pro2.iniciarHistorialCertificados],
  consulta: [Pro2.iniciarConsulta]
};

document.addEventListener("DOMContentLoaded", () => {
  Pro2.iniciarNavegacion();
  const acciones = (paginas[document.body.dataset.pagina] ?? []).filter(Boolean);
  acciones.forEach((accion) => accion());
});
```

Qué ocurre al abrir una página



> **Error frecuente:** si una página no hace nada, revisa primero que el valor de `data-pagina` coincida exactamente con una clave de `app.js`. También revisa que el script del módulo esté incluido antes de `app.js`.

SECCIÓN 04

El sistema de datos con atributos `data-*`

Los atributos `data-*` son la columna vertebral del proyecto. Se usan como ganchos para JavaScript, como almacenamiento de datos de demostración y como configuración de cada página.

Cómo se leen desde JavaScript

Un atributo con guiones se convierte en una propiedad `dataset` en formato camelCase:

```
data-usuario-id      → elemento.dataset.usuarioId
data-dosis-esquema   → elemento.dataset.dosisEsquema
data-consultorio     → elemento.dataset.consultorio
data-campo-fecha     → elemento.dataset.campoFecha
```

Los atributos se pueden seleccionar con selectores CSS:

```
const formulario = document.querySelector("[data-form-vacunacion]");
const campo = formulario.querySelector("[data-campo-fecha]");
```

Los tres usos principales

1. Ganchos de comportamiento

No contienen datos de negocio: sirven para encontrar elementos y conectar eventos.

`data-form-*` `data-campo-*` `data-tabla-*` `data-stat-*` `data-salir` `data-tema`

Ejemplo: `[data-form-vacunacion]`, `[data-campo-vacuna]`, `[data-tabla-historial]`.

2. Datos de demostración

Están dentro de contenedores ocultos y representan registros completos.

`data-datos-usuarios` `data-datos-registros` `data-datos-vacunas` `data-datos-centros` `data-datos-certificados`

Ejemplo: un `<article data-usuario>` con atributos de nombre, cédula, centro y datos de contacto. El peso y la altura se leen de cada registro de dosis.

3. Configuración de página

Indican contexto, comportamiento o alcance de la vista.

`data-pagina` `data-inicio` `data-theme` `data-alcance` `data-placeholder`

Ejemplo: `data-alcance="red"` en la consulta del Master.

4. Estado de controles

Algunos atributos se agregan o actualizan desde JavaScript.

`data-tema-listo` `data-usuario-id` `data-lote`

No siempre están escritos en el HTML inicial; pueden aparecer después de una acción.

Los bloques ocultos de datos

Los datos de demostración están en elementos con el atributo `hidden`. No se ven en pantalla, pero siguen en el DOM y `datos.js` los puede leer.

Bloque	Registro	Campos principales	Lector
`[data-datos-usuarios]`	`[data-usuario]`	id, nombre, cédula, nacimiento, sexo, región, centro, teléfono, correo, sangre, dirección, alergias, registrado por.	`leerUsuarios()`
`[data-datos-registros]`	`[data-registro]`	fecha, vacuna, tipo, procedencia, lote y vencimiento, dosis, cantidad, peso, altura, usuario, cédula, centro, aplicador, atención.	`leerRegistros()`
`[data-datos-vacunas]`	`[data-vacuna]` + `[data-lote]`	id, nombre, tipo, procedencia, laboratorio, descripción, esquema, refuerzo, cobertura y lotes.	`leerVacunas()`
`[data-datos-centros]`	`[data-centro]`	id, nombre, tipo de centro, autorización, estado, municipio, parroquia, dirección, teléfono, región.	`leerCentros()`
`[data-datos-certificados]`	`[data-certificado]` + `[data-examen-tipo]` + `[data-documento]` + `[data-dosis-respaldo]`	código, usuario, estado, emisión, vigencia, examen y centro emisor, bioanalista, médico, sello, firma, exámenes con fecha/centro, documentación y dosis de respaldo.	`leerCertificados()`
`[data-datos-personal]`	`[data-personal]`	nombre, cargo, centro.	Lectura directa en `centro.js`.
`[data-cedulas-registradas]`	`[data-cedula]`	Cédulas ya registradas para evitar duplicados.	Lectura directa en `centro.js`.
`[data-centro-actual]`	—	Centro de la sesión: id, nombre, estado, municipio, región, dirección, teléfono.	Lectura directa en `centro.js`.

Ejemplo de usuario

```
<article data-usuario
  data-usuario-id="p1"
  data-nombre="Ana Torres"
  data-cedula="V-10234567"
  data-nacimiento="1991-04-18"
  data-sexo="Femenino"
  data-region="Capital"
  data-centro-id="c1"
  data-centro="Centro de Vacunación Libertador"></article>
```

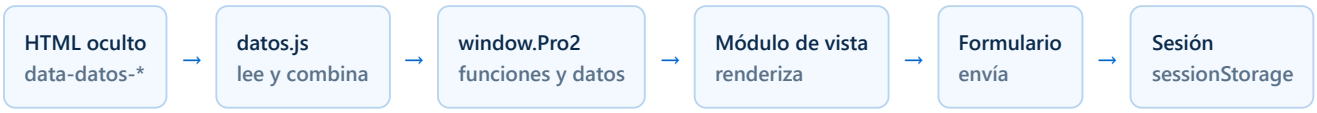
Ejemplo de vacuna con lotes

```
<article data-vacuna
  data-vacuna-id="v-covid"
  data-nombre="COVID-19"
  data-tipo="ARNm"
  data-procedencia="Internacional"
  data-laboratorio="Pfizer-BioNTech"
  data-dosis-esquema="2"
  data-refuerzo-meses="12"
  data-cobertura="Capital,Central,Occidente">
  <span data-lote data-codigo="COV-2601" data-vence="2026-12-31"></span>
  <span data-lote data-codigo="COV-2602" data-vence="2027-03-31"></span>
</article>
```

Ejemplo de certificado con exámenes

```
<article data-certificado
  data-codigo="CERT-2026-0002"
  data-usuario-id="p1"
  data-estado="Vigente"
  data-emision="2026-02-10"
  data-vigencia="2027-02-10"
  data-examen-fecha="2026-02-08"
  data-centro="Centro de Vacunación Libertador"
  data-centro-id="c1"
  data-centro-direccion="Av. Sucre, Caracas"
  data-centro-telefono="0212-5550101"
  data-centro-municipio="Libertador"
  data-centro-estado="Distrito Capital"
  data-usuario-nombre="Ana Torres"
  data-usuario-cedula="V-10234567"
  data-usuario-municipio="Libertador"
  data-usuario-estado="Distrito Capital">
  <span data-examen-tipo
    data-nombre="Hematología"
    data-descripcion="Hemograma completo"
    data-resultado="14.2"
    data-unidad="g/dL"
    data-referencia="12.0 - 16.0"
    data-fecha="2026-02-08"
    data-centro="Centro de Vacunación Libertador"></span>
  <span data-documento
    data-nombre="Informe-hematologia.pdf"
    data-tipo="application/pdf"
    data-tamano="245760"></span>
  <span data-dosis-respaldo
    data-vacuna="COVID-19"
    data-dosis="Dosis 2 de 2"
    data-fecha="2024-04-09"
    data-lote="COV-2403"
    data-lote-vence="2025-12-31"
    data-centro="Centro de Vacunación Libertador"
    data-aplicador="Dra. Elena Pérez"
    data-cargo="Médico vacunador"></span>
</article>
```

El flujo completo de los datos



Muy importante: cada página HTML es independiente. Si una página necesita usuarios, vacunas o certificados, debe incluir su propio bloque `data-datos-*`. No existe un archivo global de datos que todas las páginas compartan. Por eso los mismos datos aparecen duplicados en varias vistas.

SECCIÓN 05

La carpeta `src`: JavaScript y CSS

La carpeta `src` contiene el código fuente. `src/js` tiene el comportamiento y `src/css` tiene la fuente de los estilos.

Archivos de `src/js`

Archivo	Responsabilidad	Funciones principales
`nucleo.js`	Base compartida: constantes, tema claro/oscuro, utilidades, sesión y alertas.	`$`, `$$`, `esc`, `normalizar`, `cedulaValida`, `fechaCorta`, `calcularEdad`, `leerSesion`, `modificarSesion`, `iniciarTema`.
`datos.js`	Lee los datos de demostración del HTML y los combina con los datos nuevos de la sesión.	`leerRegistros`, `leerUsuarios`, `leerVacunas`, `leerCentros`, `leerCertificados`.
`comun.js`	Componentes compartidos de ficha de usuario, esquema de vacunación y certificado.	`esquemaUsuario`, `vacunasObligatoriasFaltantes`, `htmlFichaUsuario`, `htmlResumenEsquema`, `htmlCertificado`.
`publico.js`	Navegación general, botón de tema, salida, login de boceto, verificación y recuperación.	`iniciarNavegacion`, `iniciarLogin`, `iniciarVerificar`, `iniciarRecuperar`.
`historial.js`	Historial de vacunaciones con filtros, estadísticas y tabla; gestión de centros.	`iniciarHistorial`, `iniciarCentros`.
`consulta.js`	Consulta de perfil por cédula, limitada por `data-alcance`.	`iniciarConsulta`.
`certificados.js`	Registro y emisión de certificados médicos; historial de certificados otorgados.	`iniciarCertificados`, `iniciarHistorialCertificados`.
`centro.js`	Registro de usuarios y registro de vacunación del Perfil 1.	`iniciarRegistroUsuario`, `iniciarVacunacion`.
`avisos.js`	Preparación de avisos, selección de destinatarios y enlace de correo.	`iniciarAvisos`.
`app.js`	Arranque: navegación y despacho del inicializador según `data-pagina`.	Mapa `paginas` y `DOMContentLoaded`.

Cómo se publican las funciones

Cada archivo es una IIFE (función autoejecutable) que agrega sus funciones a `window.Pro2`. Así los demás módulos las pueden usar sin importaciones:

```
(function () {
  "use strict";

  const { $, esc } = window.Pro2;

  function hacerAlgo() {
    // ...
  }

  Object.assign(window.Pro2, {
    hacerAlgo
  });
})();
```

El archivo CSS fuente

`src/css/estilos-fuente.css` es la fuente que se compila. Contiene:

- La importación de Tailwind CSS y el plugin de daisyUI.
- Los temas `salud` (claro) y `saluddark` (oscuro).
- La tipografía Inter Variable.
- Los iconos SVG convertidos en máscaras CSS (`.icono-*`).
- Estilos base, de componentes, de la barra superior y de impresión.
- La regla que mantiene el certificado con fondo claro en modo oscuro.

El resultado compilado se guarda en `assets/css/estilos.css`. Las páginas cargan ese archivo compilado, no el fuente.

> **Comandos:** desde la raíz del proyecto, `npm run css` compila una vez y `npm run css:watch` queda observando cambios. Si agregas clases nuevas de Tailwind en HTML o JavaScript, debes recompilar.

SECCIÓN 06

Cómo modificar apartados del código

Esta sección reúne los cambios más frecuentes y el orden recomendado para hacerlos sin romper la lectura de datos ni la inicialización de la vista.

6.1 Cambiar un texto o una etiqueta

1. Busca el texto en el HTML de la vista.
2. Si el texto se genera desde JavaScript, edita la plantilla correspondiente en `src/js/`.
3. Si el texto está en una alerta o mensaje, revisa el módulo que llama a `mostrarAlerta()` o `htmlAlerta()`.

> Los textos visibles suelen estar en HTML; los mensajes dinámicos están en JavaScript.

6.2 Agregar un campo a un formulario

Ejemplo real: peso y altura en el formulario de vacunación.

1. **HTML:** agrega el campo con un `id`, un `name` y, si el JavaScript lo necesita, un atributo `data-*`.
2. **JavaScript:** obtén el elemento con `$("[data-campo-peso]")` y léelo en el `submit`.
3. **Validación:** agrega las reglas antes de guardar.
4. **Persistencia:** guarda el valor en el objeto o en la sesión con `modificarSesion()`.
5. **Lectura:** si el dato debe recuperarse en otra página, agrégalo al lector de `datos.js`.
6. **Visualización:** agrega el dato a la ficha, tabla o tarjeta correspondiente.
7. **Estilos:** si usaste clases nuevas de Tailwind, ejecuta `npm run css`.

```
<!-- HTML -->
<div class="flex flex-col gap-1.5">
  <label for="vacunacionPeso">Peso (kg)</label>
  <input id="vacunacionPeso" type="number" data-campo-peso />
</div>

// JavaScript
const campoPeso = $("[data-campo-peso]", contenedor);
const peso = campoPeso?.value.trim() ?? "";

if (!peso || Number(peso) <= 0 || Number(peso) > 500) {
  errores.push("El peso es obligatorio y debe estar entre 1 y 500 kg.");
}
```

6.3 Agregar o modificar datos de demostración

1. Localiza el bloque oculto correcto: `[data-datos-usuarios]`, `[data-datos-vacunas]`, etc.
2. Agrega un registro con un `id` único y todos los atributos que espera el lector.
3. Si agregas un campo nuevo, actualiza el mapeo de `datos.js`.
4. Duplica el bloque en todas las páginas que deban ver ese dato.
5. Verifica que los nombres de atributos estén en kebab-case: `data-dosis-esquema`, no `data-dosisEsquema`.

```
<article data-vacuna
  data-vacuna-id="v-varicela"
  data-nombre="Varicela"
  data-tipo="Virus atenuado"
  data-procedencia="Internacional"
  data-dosis-esquema="2"
  data-cobertura="Capital,Central">
  <span data-lote data-codigo="VAR-2601" data-vence="2027-12-31"></span>
</article>
```

6.4 Crear una vista nueva

1. Crea un archivo HTML dentro de la carpeta del perfil.
2. Copia la cabecera y la navegación de una vista existente del mismo perfil.
3. Incluye los scripts en el orden correcto, terminando con `app.js`.
4. Asigna un valor único a `<body data-pagina="mi-vista">`.
5. Agrega el inicializador en el mapa de `app.js`.
6. Agrega el enlace en la navegación de las vistas del perfil.
7. Incluye los bloques `data-datos-*` que necesite la vista.
8. Actualiza `index.html`, `README.md` e `INSTRUCCIONES.md` si corresponde.

```
// app.js
const paginas = {
  // ...
  "master-mi-vista": [Pro2.iniciarMiVista]
};
```

6.5 Crear un módulo JavaScript nuevo

```
(function () {
  "use strict";

  const { $, $$, esc, mostrarAlerta } = window.Pro2;

  function iniciarMiVista() {
    const contenedor = $("[data-mi-vista]");
    if (!contenedor) return;
    // lógica de la vista
  }

  Object.assign(window.Pro2, {
    iniciarMiVista
  });
})();
```

Después incluye el script antes de `app.js` y registra el inicializador en el mapa de páginas.

6.6 Agregar un filtro al historial

1. Agrega el control con `data-filtro="nombreDelFiltro"` o `data-filtro-multiple="campo"`.
2. Agrega la propiedad al objeto `filtros` de `historial.js`.
3. Inclúyela en `coincide()` para que afecte la tabla.
4. Inclúyela en el botón “Limpiar filtros”.
5. Si es selección múltiple, usa `opcionesMultiples()`.

6.7 Cambiar colores, tema o tipografía

1. Edita `src/css/estilos-fuente.css`.
2. Para el tema claro, modifica el bloque `name: "salud"`.
3. Para el tema oscuro, modifica `name: "saluddark"`.
4. Ejecuta `npm run css`.
5. Recarga la página con `Ctrl` + `F5`.

```
@plugin "daisyui/theme" {
  name: "salud";
  default: true;
  color-scheme: light;
  --color-primary: #0066cc;
  --color-base-100: #ffffff;
  --color-base-content: #2c3e50;
}
```

6.8 Agregar un icono

Los iconos son SVG en base64 usados como máscaras CSS. Se declaran en `src/css/estilos-fuente.css`:

```
.icono-nuevo {
  --icono-url: url("...");
}
```

Luego se usa en HTML:

```
<span class="icono icono-nuevo size-5" aria-hidden="true"></span>
```

6.9 Agregar un archivo descargable

1. Coloca el archivo en `assets/`.
2. Enlázalo con una ruta relativa correcta.
3. Usa el atributo `download` si debe descargarse en lugar de abrirse.

```
<a href="../assets/afiche-certificado-medico.svg"
   download="afiche-certificado-medico.svg">
  Descargar afiche
</a>
```

SECCIÓN 07

Sesión, persistencia y modelo de datos

La aplicación no tiene backend. Los cambios de la sesión se guardan en `sessionStorage` y, como respaldo para `file://`, en `window.name`.

Funciones de sesión

```
leerSesion()       // devuelve el objeto de sesión o null
guardarSesion(obj) // guarda el objeto completo
modificarSesion(fn)// lee, modifica con fn y vuelve a guardar
cerrarSesion()     // elimina la sesión
```

Claves de sesión

Clave	Contenido	Dónde se usa
`usuariosNuevos`	Usuarios creados desde el formulario.	`centro.js` y `datos.js`.
`usuariosMedidas`	Medidas heredadas de versiones anteriores. Las medidas nuevas se guardan dentro de cada dosis, en `registrosNuevos`.	`datos.js`.
`registrosNuevos`	Vacunaciones aplicadas.	`centro.js` y `datos.js`.
`centrosNuevos`	Centros creados desde el Master.	`historial.js` y `datos.js`.
`centrosEliminados`	Ids de centros eliminados.	`historial.js` y `datos.js`.
`certificadosNuevos`	Certificados emitidos.	`certificados.js` y `datos.js`.

Cómo se combinan los datos

```
// datos.js: ejemplo simplificado
const base = $$("[data-datos-usuarios] [data-usuario]").map(...);
const sesion = leerSesion();
const nuevos = sesion?.usuariosNuevos ?? [];
const medidas = sesion?.usuariosMedidas ?? {};

const combinar = (usuario) => ({
  ...usuario,
  ...(medidas[usuario.id] ?? {})
});

return base.map(combinar).concat(nuevos.map(combinar));
```

El resultado es siempre una lista combinada: primero los datos de demostración y luego los datos nuevos de la sesión. El peso y la altura de cada aplicación viajan dentro del registro de la dosis.

> **El tema** se guarda aparte en `localStorage` con la clave `pro2-tema`, porque debe conservarse entre páginas y visitas. Los datos de negocio usan `sessionStorage`.

Qué se pierde y qué se conserva

- Al cambiar de página dentro de la misma pestaña, la sesión se conserva.
- Al cerrar la pestaña o el navegador, la sesión se pierde.
- El tema claro/oscuro se conserva en `localStorage`.
- Los datos de demostración siempre vuelven al abrir de nuevo.

SECCIÓN 08

Relación con `esquema.sql`

`esquema.sql` es el esquema relacional del sistema, pensado para PostgreSQL. No está conectado a la interfaz: es la definición de la base de datos que se usaría en una versión con backend.

Entidad	Columnas clave	Relaciones
`centros`	`id`, `name`, `tipo_centro`, `autorizado`, estado, municipio, parroquia, dirección, teléfono	La subregión es un centro con `tipo_centro = 1`. Cada persona y cada dosis se vinculan al centro donde se atienden.
`personas`	`id`, `primer_nombre`, `segundo_nombre`, `primer_apellido`, `segundo_apellido`, cédula, nacimiento, sexo, teléfono, correo, región, dirección, tipo_sangre, alergias, centro_id, registrado_por	Pertenece a un centro y, a través de él, a una subregión.
`vacunas`	`id`, nombre, tipo, procedencia, laboratorio, descripción, refuerzo_meses, obligatoria, dosis_obligatorias, lotes	Catálogo de vacunas. `dosis.vacuna_id` referencia la vacuna aplicada; `lotes` guarda los lotes disponibles en el catálogo.
`dosis`	`id`, persona_id, centro_id, vacuna_id, vacuna, tipo_vacuna, procedencia, lote, lote_vence, cantidad, peso, altura, dosis, fecha, aplicador, cargo_aplicador, atención	Registra cada vacunación aplicada a una persona en un centro. `peso` y `altura` son las medidas tomadas en esa aplicación y con ellas se calcula `cantidad`. `vacuna_id` apunta al catálogo; `vacuna`, `tipo_vacuna` y `procedencia` conservan la copia del momento de la aplicación.
`certificados_salud`	`id`, código, persona_id, centro_id, dosis_id, examen_id, estado, emisión, vigencia, examen_fecha, bioanalista, médico, emisor, observaciones, sello, firma, documentación	Se relaciona con `personas`, `centros`, `dosis` (dosis principal) y `examenes` (examen principal). El detalle completo de exámenes queda en `examenes.certificado_id`.
`examenes`	`id`, certificado_id, centro_id, fecha, tipo, descripción, resultado, unidad, referencia	Pertenece a un certificado y a un centro; representa cada prueba realizada y guarda su fecha y resultado.

La columna `centros.tipo_centro` usa cuatro valores enteros: **1** Subregión, **2** CDI, **3** Módulo y **4** Hospital. La columna `centros.autorizado` indica con `TRUE` o `FALSE` si el centro está autorizado para operar.

Los cuatro campos de nombre de `personas` (`primer_nombre`, `segundo_nombre`, `primer_apellido` y `segundo_apellido`) son obligatorios en todos los formularios de registro. La columna `registrado_por` guarda quién realizó el registro.

Todas las columnas de datos de las seis entidades son obligatorias (`NOT NULL`). La FK del examen principal es diferida (`DEFERRABLE INITIALLY DEFERRED`), porque `certificados_salud` y `examenes` se referencian mutuamente. Los centros con registros asociados no se pueden eliminar mientras esos registros no se reasignen.

La entidad `vacunas` define cada vacuna del catálogo: nombre, tipo, procedencia, laboratorio, descripción, meses para el refuerzo, obligatoriedad, dosis obligatorias y lotes disponibles. `dosis.vacuna_id` la referencia y los campos de texto de `dosis` conservan la copia histórica del momento de la aplicación.

La columna `certificados_salud.documentacion` exige un arreglo JSON con al menos un documento, y la columna `dosis.lote_vence` no puede ser anterior a la fecha de aplicación.

El peso y la altura se guardan en cada fila de `dosis`, no en `personas`, porque corresponden al momento de la aplicación. La cantidad aplicada se obtiene con la superficie corporal de Mosteller: `√(peso × altura / 3600)`, escalada

respecto a `1,73 m²`. El aplicador escribe la cantidad de referencia y el formulario muestra la cantidad calculada antes de guardar.

En una versión con backend, los lectores de `datos.js` se reemplazarían por llamadas a una API y los formularios enviarían los datos al servidor. La estructura de las vistas y los atributos `data-*` podría mantenerse casi igual.

La interfaz estática conserva algunos atributos de demostración que no existen como columnas en `esquema.sql`: `data-dosis-esquema`, `data-cobertura` y `data-registrado-por-cedula`. Son metadatos de las vistas de ejemplo; la base de datos no los almacena.

SECCIÓN 09

Errores frecuentes y lista de verificación

Antes de dar por terminado un cambio, revisa esta lista. La mayoría de los problemas del proyecto vienen de la carga de scripts, de datos que faltan en la página o de clases de Tailwind sin compilar.

Errores frecuentes

Síntoma	Causa probable	Solución
La página no reacciona.	`data-pagina` no coincide con `app.js` o falta el script del módulo.	Revisa el valor del body y el orden de scripts.
No aparecen datos.	Falta el bloque `data-datos-*` en esa página.	Duplica el bloque oculto en la página.
Un campo nuevo no se guarda.	No se leyó en el `submit` o no se agregó al objeto/sesión.	Agrega la lectura, validación y persistencia.
El dato se guarda pero no se ve en otra vista.	Falta agregarlo a `datos.js` o a la función de renderizado.	Actualiza el lector y la plantilla visual.
Los estilos nuevos no se aplican.	No se recompiló el CSS.	Ejecuta `npm run css`.
Los cambios desaparecen al recargar.	Se guardaron solo en memoria y no en la sesión.	Usa `modificarSesion()`.
Una imagen o un enlace no carga.	Ruta relativa incorrecta.	Revisa `../` según la profundidad de la página.
Se ve HTML roto o texto raro.	Datos sin escapar en una plantilla.	Usa `esc()` antes de interpolar.

Lista de verificación

- El HTML tiene etiquetas `label` asociadas a sus campos.
- Los `id` son únicos dentro de la página.
- Los nombres de atributos `data-*` están en kebab-case.
- El body tiene el `data-pagina` correcto.
- El módulo de la vista está incluido antes de `app.js`.
- Los bloques de datos necesarios están en esa página.
- Los datos nuevos se guardan con `modificarSesion()`.
- Los textos dinámicos usan `esc()`.
- Se ejecutó `npm run css` si se agregaron clases.
- Se probó en modo claro y oscuro.
- Se probó en pantalla grande y en móvil.
- No hay errores en la consola del navegador.

SECCIÓN 10

Referencia rápida

Archivo → responsabilidad

Archivo o carpeta	Responsabilidad
`index.html`	Entrada, login de demostración y accesos a perfiles.
`verificar.html`	Verificación pública de certificados.
`master/`	Vistas del Perfil 3.
`subregion/`	Historial y registro de certificados del Perfil 2.
`centro/`	Registro de usuarios y vacunación del Perfil 1.
`src/js/nucleo.js`	Utilidades, sesión, tema y alertas.
`src/js/datos.js`	Lectura y combinación de datos.
`src/js/app.js`	Despacho de la vista por `data-pagina`.
`src/css/estilos-fuente.css`	Fuente de estilos y temas.
`assets/css/estilos.css`	CSS compilado que usan las páginas.
`esquema.sql`	Modelo relacional PostgreSQL.

Atributos más usados

Atributo	Uso
`data-pagina`	Indica el inicializador de la vista.
`data-inicio`	Página a la que vuelve “Salir”.
`data-theme`	Tema activo: `salud` o `saluddark`.
`data-form-*`	Formulario que un módulo debe controlar.
`data-campo-*`	Campo concreto dentro de un formulario.
`data-datos-*`	Contenedor oculto de datos de demostración.
`data-usuario`, `data-registro`, `data-vacuna`, `data-centro`, `data-certificado`	Registros de datos.
`data-filtro`, `data-filtro-multiple`	Controles del historial.
`data-alcance`	Limita la consulta a la red o a la subregión.
`data-salir`	Botón que vuelve al inicio.
`data-tema`	Botón de cambio de tema.

Comandos

```
npm install       # instala dependencias
npm run css       # compila Tailwind + daisyUI una vez
npm run css:watch # recompila al guardar cambios
```

> **Recomendación de trabajo:** antes de modificar, identifica si el cambio pertenece a HTML (estructura/texto), JS (comportamiento) o CSS (apariencia). Después revisa si el dato atraviesa `datos.js` y la sesión. Esa separación es la que mantiene el proyecto ordenado y fácil de ampliar.
